

NiceGUI testing.user_plugin 插件详细解析

`nicegui.testing.user_plugin` 是 NiceGUI 官方为**自动化测试**设计的核心插件，基于 Playwright 封装，提供了简洁、贴合 NiceGUI 组件特性的 API，用于模拟用户交互（如点击、输入、选择）、验证 UI 状态，解决了直接使用 Playwright 测试 NiceGUI 应用时“组件定位复杂、交互逻辑不匹配”的问题。以下从插件定位、安装、核心 API、使用流程、进阶场景等维度全面拆解。

一、核心定位与设计初衷

NiceGUI 应用基于 Web 技术（Vue + FastAPI），传统测试需借助 Playwright/Selenium 等工具，但直接使用这些工具存在痛点：

- 需手动通过 CSS/XPath 定位组件，而 NiceGUI 组件无固定标识，定位易失效；
- 无法直接关联 NiceGUI 组件对象与页面元素；
- 交互逻辑（如按钮点击、输入框赋值）需适配 Web 前端的异步行为。

`testing.user_plugin` 针对上述问题做了深度封装：

- 为 NiceGUI 组件自动生成唯一测试标识（无需手动设置）；
- 提供与组件类型匹配的交互 API（如 `click(button)`、`type(input, text)`）；
- 内置等待机制，自动适配异步交互（如等待后台任务完成、UI 状态更新）；
- 兼容 Playwright 原生 API，可灵活扩展。

二、前置准备

1. 安装依赖

需安装 NiceGUI 及测试相关依赖（Playwright 会被自动安装）：

```
# 安装 NiceGUI（含测试插件）
pip install nicegui[test]

# 初始化 Playwright 浏览器驱动（首次使用需执行）
playwright install
```

2. 测试文件结构

测试文件需遵循 Python 测试框架规范（如 `pytest`），典型结构：

```
my_app/
├── app.py           # NiceGUI 应用源码
└── test_app.py      # 测试用例（使用 user_plugin）
```

三、核心使用流程

1. 基础示例（应用源码 `app.py`）

```
# app.py
from nicegui import ui

def main():
    # 定义组件（无需手动设置测试标识）
    input_box = ui.input(label="输入框").bind_value_to(ui.state, "input_text")
    button = ui.button("提交", on_click=lambda: ui.notify(f"输入: {ui.state.input_text}"))
    result_label = ui.label("").bind_text_from(ui.state, "input_text")

    ui.run(native=False, show=False) # 测试时禁用原生窗口、自动打开浏览器

if __name__ == "__main__":
    main()
```

2. 测试用例（`test_app.py`）

```
# test_app.py
import pytest
from nicegui import testing
from nicegui.testing import user_plugin
from app import main

# 启动应用并注入测试插件
@pytest.fixture(scope="module")
def page():
    # 启动 NiceGUI 应用，绑定 Playwright 页面
    with testing.run_app(main) as page:
        yield page

def test_input_and_click(page: user_plugin.Page):
    # 1. 模拟用户在输入框输入文本
    input_box = page.get_by_label("输入框") # 通过标签定位输入框组件
    page.type(input_box, "Hello NiceGUI") # 模拟输入

    # 2. 模拟点击提交按钮
    button = page.get_by_text("提交") # 通过文本定位按钮组件
    page.click(button) # 模拟点击

    # 3. 验证结果标签的文本
    result_label = page.get_by_text("Hello NiceGUI")
    assert result_label.exists() # 断言标签存在且文本正确

    # 4. 验证通知提示
    notification = page.get_by_text("输入: Hello NiceGUI")
    assert notification.exists()
```

3. 运行测试

```
pytest test_app.py -v
```

四、核心 API 详解

`user_plugin` 封装的 API 分为**组件定位**、**用户交互**、**状态验证**三类，核心方法如下：

类别	方法	作用	示例
组件定位	<code>get_by_label(label)</code>	通过组件 label 定位（适用于输入框、选择器等）	<code>page.get_by_label("用户名")</code>
	<code>get_by_text(text)</code>	通过文本内容定位（适用于按钮、标签、通知）	<code>page.get_by_text("提交")</code>
	<code>get_by_component(component)</code>	直接通过 NiceGUI 组件对象定位（需提前引用）	<code>page.get_by_component(button)</code>
用户交互	<code>click(element)</code>	模拟鼠标点击组件	<code>page.click(button)</code>
	<code>type(element, text)</code>	模拟键盘输入文本（适用于输入框、文本域）	<code>page.type(input, "123456")</code>
	<code>clear(element)</code>	清空输入框内容	<code>page.clear(input)</code>
	<code>select(element, value)</code>	模拟选择下拉框选项	<code>page.select(select_box, "选项2")</code>
	<code>check(element)</code>	模拟勾选复选框	<code>page.check(checkbox)</code>
状态验证	<code>exists(element)</code>	验证组件是否存在于页面	<code>assert page.get_by_text("成功").exists()</code>
	<code>inner_text(element)</code>	获取组件的文本内容	<code>assert page.inner_text(label) == "结果"</code>
	<code>wait_for(element)</code>	等待组件出现（超时抛出异常）	<code>page.wait_for(page.get_by_text("加载完成"))</code>

五、进阶使用场景

1. 测试后台任务

`user_plugin` 内置等待机制，可等待后台任务完成后验证 UI 状态：

```
# app.py 新增后台任务逻辑
from nicegui import ui, background_tasks
import asyncio

async def long_task(text: str):
    await asyncio.sleep(2)
    ui.state.result = f"处理完成: {text}"

def main():
    input_box = ui.input(label="输入")
```

```

        button = ui.button("处理", on_click=lambda:
background_tasks.create(long_task(input_box.value)))
        result_label = ui.label("").bind_text_from(ui.state, "result")
        ui.run(native=False, show=False)

# test_app.py 测试用例
def test_background_task(page: user_plugin.Page):
    # 输入文本并点击按钮
    page.type(page.get_by_label("输入"), "测试任务")
    page.click(page.get_by_text("处理"))

    # 等待后台任务完成，验证结果
    result_label = page.get_by_text("处理完成: 测试任务")
    page.wait_for(result_label, timeout=3000) # 超时3秒
    assert result_label.exists()

```

2. 测试弹窗 / 对话框

模拟对话框的确认 / 取消操作:

```

# app.py 对话框逻辑
from nicegui import ui

def main():
    def open_dialog():
        with ui.dialog() as dialog, ui.card():
            ui.label("确认删除? ")
            ui.button("确认", on_click=lambda: (dialog.close(), ui.notify("已删除")))
            ui.button("取消", on_click=dialog.close)
        dialog.open()

    ui.button("删除", on_click=open_dialog)
    ui.run(native=False, show=False)

# test_app.py 测试用例
def test_dialog(page: user_plugin.Page):
    # 打开对话框
    page.click(page.get_by_text("删除"))
    # 点击“确认”按钮
    page.click(page.get_by_text("确认"))
    # 验证通知
    assert page.get_by_text("已删除").exists()

```

3. 兼容 Playwright 原生 API

`user_plugin.Page` 继承自 Playwright 的 `Page` 对象，可直接调用原生 API 扩展测试逻辑:

```
def test_native_api(page: user_plugin.Page):
    # 使用 user_plugin API 定位组件
    input_box = page.get_by_label("输入框")
    page.type(input_box, "测试")

    # 使用 Playwright 原生 API 验证 DOM 状态
    native_input = page.locator(f'[data-testid="{input_box.test_id}"]')
    assert native_input.input_value() == "测试"
```

六、关键注意事项

1. 应用启动配置

测试时需调整 `ui.run()` 参数，避免干扰自动化测试：

```
ui.run(
    native=False,      # 禁用原生窗口（使用 Playwright 控制浏览器）
    show=False,        # 不自动打开浏览器（由 playwright 接管）
    reload=False,      # 禁用热重载（避免测试中应用重启）
    port=8080          # 固定端口（便于 playwright 连接）
)
```

2. 组件定位优先级

推荐定位方式优先级：

- 1. `get_by_component(component)`（最稳定，需提前引用组件对象）；
- 2. `get_by_label(label)`（适用于带 label 的组件）；
- 3. `get_by_text(text)`（适用于文本类组件，注意文本唯一性）；
- 4. Playwright 原生 `locator()`（兜底方案）。

3. 超时与等待

- 所有交互 API 内置默认超时（5 秒），可通过 `timeout` 参数调整：`page.click(button, timeout=10000)`；
- 对于异步操作（如后台任务、网络请求），需使用 `page.wait_for()` 显式等待。

4. 测试范围限制

- `user_plugin` 仅适用于测试 NiceGUI 前端交互，无法直接测试纯后端逻辑（需单独编写单元测试）；
- 暂不支持测试 `ui.native` 原生组件（如原生窗口、系统对话框）。

七、与纯 Playwright 测试的对比

特性	<code>testing.user_plugin</code>	纯 Playwright
组件定位	基于 NiceGUI 组件特性（label/text/ 对象），稳定	需手写 CSS/XPath，易失效
交互 API	贴合 NiceGUI 组件类型（如 <code>select</code> 适配下拉框）	通用 Web 交互 API，需适配
异步等待	内置适配 NiceGUI 异步逻辑	需手动编写等待逻辑
学习成本	低（贴合 NiceGUI 开发习惯）	高（需掌握 Web 测试知识）

特性	<code>testing.user_plugin</code>	纯 Playwright
灵活性	兼容原生 API, 可扩展	完全灵活, 但需手动适配